

1 Introduction

1.1 What is Bash?

Bash is the shell, or command language interpreter, for the GNU operating system. The name is an acronym for the ‘**B**ourne-**A**gain **S**hell’, a pun on Stephen Bourne, the author of the direct ancestor of the current Unix shell **sh**, which appeared in the Seventh Edition Bell Labs Research version of Unix.

Bash is largely compatible with **sh** and incorporates useful features from the Korn shell **ksh** and the C shell **csh**. It is intended to be a conformant implementation of the IEEE POSIX Shell and Tools portion of the IEEE POSIX specification (IEEE Standard 1003.1). It offers functional improvements over **sh** for both interactive and programming use.

While the GNU operating system provides other shells, including a version of **csh**, Bash is the default shell. Like other GNU software, Bash is quite portable. It currently runs on nearly every version of Unix and a few other operating systems – independently-supported ports exist for Windows and other platforms.

1.2 What is a shell?

At its base, a shell is simply a macro processor that executes commands. The term macro processor means functionality where text and symbols are expanded to create larger expressions.

A Unix shell is both a command interpreter and a programming language. As a command interpreter, the shell provides the user interface to the rich set of GNU utilities. The programming language features allow these utilities to be combined. Users can create files containing commands, and these become commands themselves. These new commands have the same status as system commands in directories such as **/bin**, allowing users or groups to establish custom environments to automate their common tasks.

Shells may be used interactively or non-interactively. In interactive mode, they accept input typed from the keyboard. When executing non-interactively, shells execute commands read from a file or a string.

A shell allows execution of GNU commands, both synchronously and asynchronously. The shell waits for synchronous commands to complete before accepting more input; asynchronous commands continue to execute in parallel with the shell while it reads and executes additional commands. The *redirection* constructs permit fine-grained control of the input and output of those commands. Moreover, the shell allows control over the contents of commands’ environments.

Shells also provide a small set of built-in commands (*builtins*) implementing functionality impossible or inconvenient to obtain via separate utilities. For example, **cd**, **break**, **continue**, and **exec** cannot be implemented outside of the shell because they directly manipulate the shell itself. The **history**, **getopts**, **kill**, or **pwd** builtins, among others, could be implemented in separate utilities, but they are more convenient to use as builtin commands. All of the shell builtins are described in subsequent sections.

While executing commands is essential, most of the power (and complexity) of shells is due to their embedded programming languages. Like any high-level language, the shell provides variables, flow control constructs, quoting, and functions.

Shells offer features geared specifically for interactive use rather than to augment the programming language. These interactive features include job control, command line editing, command history and aliases. This manual describes how Bash provides all of these features.

2 Definitions

These definitions are used throughout the remainder of this manual.

POSIX	A family of open system standards based on Unix. Bash is primarily concerned with the Shell and Utilities portion of the POSIX 1003.1 standard.
blank	A space or tab character.
whitespace	A character belonging to the space character class in the current locale, or for which isspace() returns true.
builtin	A command that is implemented internally by the shell itself, rather than by an executable program somewhere in the file system.
control operator	A token that performs a control function. It is a newline or one of the following: ' ', '&&', '&', ';', ';;', ';&', ';;&', ' ', ' &', '(', or ')'.
exit status	The value returned by a command to its caller. The value is restricted to eight bits, so the maximum value is 255.
field	A unit of text that is the result of one of the shell expansions. After expansion, when executing a command, the resulting fields are used as the command name and arguments.
filename	A string of characters used to identify a file.
job	A set of processes comprising a pipeline, and any processes descended from it, that are all in the same process group.
job control	A mechanism by which users can selectively stop (suspend) and restart (resume) execution of processes.
metacharacter	A character that, when unquoted, separates words. A metacharacter is a space , tab , newline , or one of the following characters: ' ', '&', ';', '(', ')', '<', or '>'.
name	A word consisting solely of letters, numbers, and underscores, and beginning with a letter or underscore. Names are used as shell variable and function names. Also referred to as an identifier .
operator	A control operator or a redirection operator . See Section 3.6 [Redirections], page 41, for a list of redirection operators. Operators contain at least one unquoted metacharacter .
process group	A collection of related processes each having the same process group ID.
process group ID	A unique identifier that represents a process group during its lifetime.

reserved word

A **word** that has a special meaning to the shell. Most reserved words introduce shell flow control constructs, such as **for** and **while**.

return status

A synonym for **exit status**.

signal

A mechanism by which a process may be notified by the kernel of an event occurring in the system.

special builtin

A shell builtin command that has been classified as special by the POSIX standard.

token

A sequence of characters considered a single unit by the shell. It is either a **word** or an **operator**.

word

A sequence of characters treated as a unit by the shell. Words may not include unquoted **metacharacters**.

3 Basic Shell Features

Bash is an acronym for ‘**B**ourne-**A**gain **S**hell’. The Bourne shell is the traditional Unix shell originally written by Stephen Bourne. All of the Bourne shell builtin commands are available in Bash, and the rules for evaluation and quoting are taken from the POSIX specification for the ‘standard’ Unix shell.

This chapter briefly summarizes the shell’s ‘building blocks’: commands, control structures, shell functions, shell *parameters*, shell expansions, *redirections*, which are a way to direct input and output from and to named files, and how the shell executes commands.

3.1 Shell Syntax

When the shell reads input, it proceeds through a sequence of operations. If the input indicates the beginning of a comment, the shell ignores the comment symbol (`#`), and the rest of that line.

Otherwise, roughly speaking, the shell reads its input and divides the input into words and operators, employing the quoting rules to select which meanings to assign various words and characters.

The shell then parses these tokens into commands and other constructs, removes the special meaning of certain words or characters, expands others, redirects input and output as needed, executes the specified command, waits for the command’s exit status, and makes that exit status available for further inspection or processing.

3.1.1 Shell Operation

The following is a brief description of the shell’s operation when it reads and executes a command. Basically, the shell does the following:

1. Reads its input from a file (see Section 3.8 [Shell Scripts], page 50), from a string supplied as an argument to the `-c` invocation option (see Section 6.1 [Invoking Bash], page 100), or from the user’s terminal.
2. Breaks the input into words and operators, obeying the quoting rules described in Section 3.1.2 [Quoting], page 6. These tokens are separated by **metacharacters**. This step performs alias expansion (see Section 6.6 [Aliases], page 109).
3. Parses the tokens into simple and compound commands (see Section 3.2 [Shell Commands], page 9).
4. Performs the various shell expansions (see Section 3.5 [Shell Expansions], page 24), breaking the expanded tokens into lists of filenames (see Section 3.5.8 [Filename Expansion], page 39) and commands and arguments.
5. Performs any necessary redirections (see Section 3.6 [Redirections], page 41) and removes the redirection operators and their operands from the argument list.
6. Executes the command (see Section 3.7 [Executing Commands], page 45).
7. Optionally waits for the command to complete and collects its exit status (see Section 3.7.5 [Exit Status], page 48).

3.1.2 Quoting

Quoting is used to remove the special meaning of certain characters or words to the shell. Quoting can be used to disable special treatment for special characters, to prevent reserved words from being recognized as such, and to prevent parameter expansion.

Each of the shell metacharacters (see Chapter 2 [Definitions], page 3) has special meaning to the shell and must be quoted if it is to represent itself.

When the command history expansion facilities are being used (see Section 9.3 [History Interaction], page 171), the *history expansion* character, usually `!`, must be quoted to prevent history expansion. See Section 9.1 [Bash History Facilities], page 168, for more details concerning history expansion.

There are four quoting mechanisms: the *escape character*, single quotes, double quotes, and dollar-single quotes.

3.1.2.1 Escape Character

A non-quoted backslash `\` is the Bash escape character. It preserves the literal value of the next character that follows, removing any special meaning it has, with the exception of **newline**. If a `\newline` pair appears, and the backslash itself is not quoted, the `\newline` is treated as a line continuation (that is, it is removed from the input stream and effectively ignored).

3.1.2.2 Single Quotes

Enclosing characters in single quotes (`'`) preserves the literal value of each character within the quotes. A single quote may not occur between single quotes, even when preceded by a backslash.

3.1.2.3 Double Quotes

Enclosing characters in double quotes (`"`) preserves the literal value of all characters within the quotes, with the exception of `$`, `'`, `\`, and, when history expansion is enabled, `!`. When the shell is in POSIX mode (see Section 6.11 [Bash POSIX Mode], page 116), the `!` has no special meaning within double quotes, even when history expansion is enabled. The characters `$` and `'` retain their special meaning within double quotes (see Section 3.5 [Shell Expansions], page 24). The backslash retains its special meaning only when followed by one of the following characters: `$`, `'`, `"`, `\`, or **newline**. Within double quotes, backslashes that are followed by one of these characters are removed. Backslashes preceding characters without a special meaning are left unmodified.

A double quote may be quoted within double quotes by preceding it with a backslash. If enabled, history expansion will be performed unless an `!` appearing in double quotes is escaped using a backslash. The backslash preceding the `!` is not removed.

The special parameters `*` and `@` have special meaning when in double quotes (see Section 3.5.3 [Shell Parameter Expansion], page 27).

3.1.2.4 ANSI-C Quoting

Character sequences of the form `$'string'` are treated as a special kind of single quotes. The sequence expands to *string*, with backslash-escaped characters in *string* replaced as

specified by the ANSI C standard. Backslash escape sequences, if present, are decoded as follows:

<code>\a</code>	alert (bell)
<code>\b</code>	backspace
<code>\e</code>	
<code>\E</code>	An escape character (not in ANSI C).
<code>\f</code>	form feed
<code>\n</code>	newline
<code>\r</code>	carriage return
<code>\t</code>	horizontal tab
<code>\v</code>	vertical tab
<code>\\</code>	backslash
<code>\'</code>	single quote
<code>\"</code>	double quote
<code>\?</code>	question mark
<code>\nnn</code>	The eight-bit character whose value is the octal value <i>nnn</i> (one to three octal digits).
<code>\xHH</code>	The eight-bit character whose value is the hexadecimal value <i>HH</i> (one or two hex digits).
<code>\uHHHH</code>	The Unicode (ISO/IEC 10646) character whose value is the hexadecimal value <i>HHHH</i> (one to four hex digits).
<code>\UHHHHHHHH</code>	The Unicode (ISO/IEC 10646) character whose value is the hexadecimal value <i>HHHHHHHH</i> (one to eight hex digits).
<code>\cx</code>	A control-x character.

The expanded result is single-quoted, as if the dollar sign had not been present.

3.1.2.5 Locale-Specific Translation

Prefixing a double-quoted string with a dollar sign (`'$'`), such as `$"hello, world"`, causes the string to be translated according to the current locale. The `gettext` infrastructure performs the lookup and translation, using the `LC_MESSAGES`, `TEXTDOMAINDIR`, and `TEXTDOMAIN` shell variables, as explained below. See the `gettext` documentation for additional details not covered here. If the current locale is `C` or `POSIX`, if there are no translations available, or if the string is not translated, the dollar sign is ignored, and the string is treated as double-quoted as described above. Since this is a form of double quoting, the string remains double-quoted by default, whether or not it is translated and replaced. If the `noexpand_translation` option is enabled using the `shopt` builtin (see Section 4.3.2 [The Shopt Builtin], page 78), translated strings are single-quoted instead of double-quoted.

The rest of this section is a brief overview of how you use `gettext` to create translations for strings in a shell script named *scriptname*. There are more details in the `gettext` documentation.

Once you've marked the strings in your script that you want to translate using `$"...`", you create a `gettext` "template" file using the command

```
bash --dump-po-strings scriptname > domain.pot
```

The *domain* is your *message domain*. It's just an arbitrary string that's used to identify the files `gettext` needs, like a package or script name. It needs to be unique among all the message domains on systems where you install the translations, so `gettext` knows which translations correspond to your script. You'll use the template file to create translations for each target language. The template file conventionally has the suffix `' .pot'`.

You copy this template file to a separate file for each target language you want to support (called "PO" files, which use the suffix `' .po'`). PO files use various naming conventions, but when you are working to translate a template file into a particular language, you first copy the template file to a file whose name is the language you want to target, with the `' .po'` suffix. For instance, the Spanish translations of your strings would be in a file named `'es.po'`, and to get started using a message domain named "example," you would run

```
cp example.pot es.po
```

Ultimately, PO files are often named *domain.po* and installed in directories that contain multiple translation files for a particular language.

Whichever naming convention you choose, you will need to translate the strings in the PO files into the appropriate languages. This has to be done manually.

When you have the translations and PO files complete, you'll use the `gettext` tools to produce what are called "MO" files, which are compiled versions of the PO files the `gettext` tools use to look up translations efficiently. MO files are also called "message catalog" files. You use the `msgfmt` program to do this. For instance, if you had a file with Spanish translations, you could run

```
msgfmt -o es.mo es.po
```

to produce the corresponding MO file.

Once you have the MO files, you decide where to install them and use the `TEXTDOMAINDIR` shell variable to tell the `gettext` tools where they are. Make sure to use the same message domain to name the MO files as you did for the PO files when you install them.

Your users will use the `LANG` or `LC_MESSAGES` shell variables to select the desired language.

You set the `TEXTDOMAIN` variable to the script's message domain. As above, you use the message domain to name your translation files.

You, or possibly your users, set the `TEXTDOMAINDIR` variable to the name of a directory where the message catalog files are stored. If you install the message files into the system's standard message catalog directory, you don't need to worry about this variable.

The directory where the message catalog files are stored varies between systems. Some use the message catalog selected by the `LC_MESSAGES` shell variable. Others create the name of the message catalog from the value of the `TEXTDOMAIN` shell variable, possibly adding the `' .mo'` suffix. If you use the `TEXTDOMAIN` variable, you may need to set the `TEXTDOMAINDIR` variable to the location of the message catalog files, as above. It's common to use both variables in this fashion: `$TEXTDOMAINDIR/$LC_MESSAGES/LC_MESSAGES/$TEXTDOMAIN.mo`.

If you used that last convention, and you wanted to store the message catalog files with Spanish (es) and Esperanto (eo) translations into a local directory you use for custom translation files, you could run

```
TEXTDOMAIN=example
TEXTDOMAINDIR=/usr/local/share/locale

cp es.mo ${TEXTDOMAINDIR}/es/LC_MESSAGES/${TEXTDOMAIN}.mo
cp eo.mo ${TEXTDOMAINDIR}/eo/LC_MESSAGES/${TEXTDOMAIN}.mo
```

When all of this is done, and the message catalog files containing the compiled translations are installed in the correct location, your users will be able to see translated strings in any of the supported languages by setting the `LANG` or `LC_MESSAGES` environment variables before running your script.

3.1.3 Comments

In a non-interactive shell, or an interactive shell in which the `interactive_comments` option to the `shopt` builtin is enabled (see Section 4.3.2 [The Shopt Builtin], page 78), a word beginning with ‘#’ introduces a comment. A word begins at the beginning of a line, after unquoted whitespace, or after an operator. The comment causes that word and all remaining characters on that line to be ignored. An interactive shell without the `interactive_comments` option enabled does not allow comments. The `interactive_comments` option is enabled by default in interactive shells. See Section 6.3 [Interactive Shells], page 104, for a description of what makes a shell interactive.

3.2 Shell Commands

A simple shell command such as `echo a b c` consists of the command itself followed by arguments, separated by spaces.

More complex shell commands are composed of simple commands arranged together in a variety of ways: in a pipeline in which the output of one command becomes the input of a second, in a loop or conditional construct, or in some other grouping.

3.2.1 Reserved Words

Reserved words are words that have special meaning to the shell. They are used to begin and end the shell’s compound commands.

The following words are recognized as reserved when unquoted and the first word of a command (see below for exceptions):

```
if      then    elif    else    fi      time
for     in      until   while   do      done
case    esac    coproc  select function
{       }        [[      ]]     !
```

`in` is recognized as a reserved word if it is the third word of a `case` or `select` command. `in` and `do` are recognized as reserved words if they are the third word in a `for` command.

3.2.2 Simple Commands

A simple command is the kind of command that’s executed most often. It’s just a sequence of words separated by blanks, terminated by one of the shell’s control operators (see Chapter 2

[Definitions], page 3). The first word generally specifies a command to be executed, with the rest of the words being that command's arguments.

The return status (see Section 3.7.5 [Exit Status], page 48) of a simple command is its exit status as provided by the POSIX 1003.1 `waitpid` function, or `128+n` if the command was terminated by signal `n`.

3.2.3 Pipelines

A **pipeline** is a sequence of one or more commands separated by one of the control operators `|` or `&`.

The format for a pipeline is

```
[time [-p]] [!] command1 [ | or |& command2 ] ...
```

The output of each command in the pipeline is connected via a pipe to the input of the next command. That is, each command reads the previous command's output. This connection is performed before any redirections specified by *command1*.

If `|&` is the pipeline operator, *command1*'s standard error, in addition to its standard output, is connected to *command2*'s standard input through the pipe; it is shorthand for `2>&1 |`. This implicit redirection of the standard error to the standard output is performed after any redirections specified by *command1*, consistent with that shorthand.

If the reserved word `time` precedes the pipeline, Bash prints timing statistics for the pipeline once it finishes. The statistics currently consist of elapsed (wall-clock) time and user and system time consumed by the command's execution. The `-p` option changes the output format to that specified by POSIX. When the shell is in POSIX mode (see Section 6.11 [Bash POSIX Mode], page 116), it does not recognize `time` as a reserved word if the next token begins with a `-`. The value of the `TIMEFORMAT` variable is a format string that specifies how the timing information should be displayed. See Section 5.2 [Bash Variables], page 87, for a description of the available formats. Providing `time` as a reserved word permits the timing of shell builtins, shell functions, and pipelines. An external `time` command cannot time these easily.

When the shell is in POSIX mode (see Section 6.11 [Bash POSIX Mode], page 116), you can use `time` by itself as a simple command. In this case, the shell displays the total user and system time consumed by the shell and its children. The `TIMEFORMAT` variable specifies the format of the time information.

If a pipeline is not executed asynchronously (see Section 3.2.4 [Lists], page 11), the shell waits for all commands in the pipeline to complete.

Each command in a multi-command pipeline, where pipes are created, is executed in its own *subshell*, which is a separate process (see Section 3.7.3 [Command Execution Environment], page 46). If the `lastpipe` option is enabled using the `shopt` builtin (see Section 4.3.2 [The Shopt Builtin], page 78), and job control is not active, the last element of a pipeline may be run by the shell process.

The exit status of a pipeline is the exit status of the last command in the pipeline, unless the `pipefail` option is enabled (see Section 4.3.1 [The Set Builtin], page 74). If `pipefail` is enabled, the pipeline's return status is the value of the last (rightmost) command to exit with a non-zero status, or zero if all commands exit successfully. If the reserved word `!` precedes the pipeline, the exit status is the logical negation of the exit status as described

above. If a pipeline is not executed asynchronously (see Section 3.2.4 [Lists], page 11), the shell waits for all commands in the pipeline to terminate before returning a value. The return status of an asynchronous pipeline is 0.

3.2.4 Lists of Commands

A **list** is a sequence of one or more pipelines separated by one of the operators `;`, `&`, `&&`, or `||`, and optionally terminated by one of `;`, `&`, or a **newline**.

Of these list operators, `&&` and `||` have equal precedence, followed by `;` and `&`, which have equal precedence.

A sequence of one or more newlines may appear in a **list** to delimit commands, equivalent to a semicolon.

If a command is terminated by the control operator `&`, the shell executes the command asynchronously in a subshell. This is known as executing the command in the *background*, and these are referred to as *asynchronous* commands. The shell does not wait for the command to finish, and the return status is 0 (true). When job control is not active (see Chapter 7 [Job Control], page 125), the standard input for asynchronous commands, in the absence of any explicit redirections, is redirected from `/dev/null`.

Commands separated by a `;` are executed sequentially; the shell waits for each command to terminate in turn. The return status is the exit status of the last command executed.

AND and OR lists are sequences of one or more pipelines separated by the control operators `&&` and `||`, respectively. AND and OR lists are executed with left associativity.

An AND list has the form

```
command1 && command2
```

command2 is executed if, and only if, *command1* returns an exit status of zero (success).

An OR list has the form

```
command1 || command2
```

command2 is executed if, and only if, *command1* returns a non-zero exit status.

The return status of AND and OR lists is the exit status of the last command executed in the list.

3.2.5 Compound Commands

Compound commands are the shell programming language constructs. Each construct begins with a reserved word or control operator and is terminated by a corresponding reserved word or operator. Any redirections (see Section 3.6 [Redirections], page 41) associated with a compound command apply to all commands within that compound command unless explicitly overridden.

In most cases a list of commands in a compound command's description may be separated from the rest of the command by one or more newlines, and may be followed by a newline in place of a semicolon.

Bash provides looping constructs, conditional commands, and mechanisms to group commands and execute them as a unit.

- Equivalent to `--`.

A *login shell* is one whose first character of argument zero is `'-'`, or one invoked with the `--login` option.

An *interactive shell* is one started without non-option arguments, unless `-s` is specified, without specifying the `-c` option, and whose standard input and standard error are both connected to terminals (as determined by *isatty(3)*), or one started with the `-i` option. See Section 6.3 [Interactive Shells], page 104, for more information.

If arguments remain after option processing, and neither the `-c` nor the `-s` option has been supplied, the first argument is treated as the name of a file containing shell commands (see Section 3.8 [Shell Scripts], page 50). When Bash is invoked in this fashion, `$0` is set to the name of the file, and the positional parameters are set to the remaining arguments. Bash reads and executes commands from this file, then exits. Bash's exit status is the exit status of the last command executed in the script. If no commands are executed, the exit status is 0. Bash first attempts to open the file in the current directory, and, if no file is found, searches the directories in `PATH` for the script.

6.2 Bash Startup Files

This section describes how Bash executes its startup files. If any of the files exist but cannot be read, Bash reports an error. Tildes are expanded in filenames as described above under Tilde Expansion (see Section 3.5.2 [Tilde Expansion], page 26).

Interactive shells are described in Section 6.3 [Interactive Shells], page 104.

Invoked as an interactive login shell, or with `--login`

When Bash is invoked as an interactive login shell, or as a non-interactive shell with the `--login` option, it first reads and executes commands from the file `/etc/profile`, if that file exists. After reading that file, it looks for `~/.bash_profile`, `~/.bash_login`, and `~/.profile`, in that order, and reads and executes commands from the first one that exists and is readable. The `--noprofile` option inhibits this behavior.

When an interactive login shell exits, or a non-interactive login shell executes the `exit` builtin command, Bash reads and executes commands from the file `~/.bash_logout`, if it exists.

Invoked as an interactive non-login shell

When Bash runs as an interactive shell that is not a login shell, it reads and executes commands from `~/.bashrc`, if that file exists. The `--norc` option inhibits this behavior. The `--rcfile file` option causes Bash to use *file* instead of `~/.bashrc`.

So, typically, your `~/.bash_profile` contains the line

```
if [ -f ~/.bashrc ]; then . ~/.bashrc; fi
```

after (or before) any login-specific initializations.

Invoked non-interactively

When Bash is started non-interactively, to run a shell script, for example, it looks for the variable `BASH_ENV` in the environment, expands its value if it appears there, and uses the

expanded value as the name of a file to read and execute. Bash behaves as if the following command were executed:

```
if [ -n "$BASH_ENV" ]; then . "$BASH_ENV"; fi
```

but does not the value of the `PATH` variable to search for the filename.

As noted above, if a non-interactive shell is invoked with the `--login` option, Bash attempts to read and execute commands from the login shell startup files.

Invoked with name `sh`

If Bash is invoked with the name `sh`, it tries to mimic the startup behavior of historical versions of `sh` as closely as possible, while conforming to the POSIX standard as well.

When invoked as an interactive login shell, or as a non-interactive shell with the `--login` option, it first attempts to read and execute commands from `/etc/profile` and `~/.profile`, in that order. The `--noprofile` option inhibits this behavior.

When invoked as an interactive shell with the name `sh`, Bash looks for the variable `ENV`, expands its value if it is defined, and uses the expanded value as the name of a file to read and execute. Since a shell invoked as `sh` does not attempt to read and execute commands from any other startup files, the `--rcfile` option has no effect.

A non-interactive shell invoked with the name `sh` does not attempt to read any other startup files.

When invoked as `sh`, Bash enters POSIX mode after reading the startup files.

Invoked in POSIX mode

When Bash is started in POSIX mode, as with the `--posix` command line option, it follows the POSIX standard for startup files. In this mode, interactive shells expand the `ENV` variable and read and execute commands from the file whose name is the expanded value. No other startup files are read.

Invoked by remote shell daemon

Bash attempts to determine when it is being run with its standard input connected to a network connection, as when executed by the historical and rarely-seen remote shell daemon, usually `rshd`, or the secure shell daemon `sshd`. If Bash determines it is being run non-interactively in this fashion, it reads and executes commands from `~/.bashrc`, if that file exists and is readable. Bash does not read this file if invoked as `sh`. The `--norc` option inhibits this behavior, and the `--rcfile` option makes Bash use a different file instead of `~/.bashrc`, but neither `rshd` nor `sshd` generally invoke the shell with those options or allow them to be specified.

Invoked with unequal effective and real UID/GIDs

If Bash is started with the effective user (group) id not equal to the real user (group) id, and the `-p` option is not supplied, no startup files are read, shell functions are not inherited from the environment, the `SHELLOPTS`, `BASHOPTS`, `CDPATH`, and `GLOBIGNORE` variables, if they appear in the environment, are ignored, and the effective user id is set to the real user id. If the `-p` option is supplied at invocation, the startup behavior is the same, but the effective user id is not reset.

6.3 Interactive Shells

6.3.1 What is an Interactive Shell?

An interactive shell is one started without non-option arguments (unless `-s` is specified) and without specifying the `-c` option, whose input and error output are both connected to terminals (as determined by `isatty(3)`), or one started with the `-i` option.

An interactive shell generally reads from and writes to a user's terminal.

The `-s` invocation option may be used to set the positional parameters when an interactive shell starts.

6.3.2 Is this Shell Interactive?

To determine within a startup script whether or not Bash is running interactively, test the value of the `'-'` special parameter. It contains `i` when the shell is interactive. For example:

```
case "$-" in
  *i*) echo This shell is interactive ;;
  *) echo This shell is not interactive ;;
esac
```

Alternatively, startup scripts may examine the variable `PS1`; it is unset in non-interactive shells, and set in interactive shells. Thus:

```
if [ -z "$PS1" ]; then
    echo This shell is not interactive
else
    echo This shell is interactive
fi
```

6.3.3 Interactive Shell Behavior

When the shell is running interactively, it changes its behavior in several ways.

1. Bash reads and executes startup files as described in Section 6.2 [Bash Startup Files], page 102.
2. Job Control (see Chapter 7 [Job Control], page 125) is enabled by default. When job control is in effect, Bash ignores the keyboard-generated job control signals `SIGTTIN`, `SIGTTOU`, and `SIGTSTP`.
3. Bash executes the values of the set elements of the `PROMPT_COMMAND` array variable as commands before printing the primary prompt, `$PS1` (see Section 5.2 [Bash Variables], page 87).
4. Bash expands and displays `PS1` before reading the first line of a command, and expands and displays `PS2` before reading the second and subsequent lines of a multi-line command. Bash expands and displays `PS0` after it reads a command but before executing it. See Section 6.9 [Controlling the Prompt], page 114, for a complete list of prompt string escape sequences.
5. Bash uses Readline (see Chapter 8 [Command Line Editing], page 130) to read commands from the user's terminal.
6. Bash inspects the value of the `ignoreeof` option to `set -o` instead of exiting immediately when it receives an EOF on its standard input when reading a command (see Section 4.3.1 [The Set Builtin], page 74).

7. Bash enables Command history (see Section 9.1 [Bash History Facilities], page 168) and history expansion (see Section 9.3 [History Interaction], page 171) by default. When a shell with history enabled exits, Bash saves the command history to the file named by `$HISTFILE`.
8. Alias expansion (see Section 6.6 [Aliases], page 109) is performed by default.
9. In the absence of any traps, Bash ignores `SIGTERM` (see Section 3.7.6 [Signals], page 49).
10. In the absence of any traps, `SIGINT` is caught and handled (see Section 3.7.6 [Signals], page 49). `SIGINT` will interrupt some shell builtins.
11. An interactive login shell sends a `SIGHUP` to all jobs on exit if the `huponexit` shell option has been enabled (see Section 3.7.6 [Signals], page 49).
12. The `-n` option has no effect, whether at invocation or when using `'set -n'` (see Section 4.3.1 [The Set Builtin], page 74).
13. Bash will check for mail periodically, depending on the values of the `MAIL`, `MAILPATH`, and `MAILCHECK` shell variables (see Section 5.2 [Bash Variables], page 87).
14. The shell will not exit on expansion errors due to references to unbound shell variables after `'set -u'` has been enabled (see Section 4.3.1 [The Set Builtin], page 74).
15. The shell will not exit on expansion errors caused by `var` being unset or null in `${var:?word}` expansions (see Section 3.5.3 [Shell Parameter Expansion], page 27).
16. Redirection errors encountered by shell builtins will not cause the shell to exit.
17. When running in POSIX mode, a special builtin returning an error status will not cause the shell to exit (see Section 6.11 [Bash POSIX Mode], page 116).
18. A failed `exec` will not cause the shell to exit (see Section 4.1 [Bourne Shell Builtins], page 52).
19. Parser syntax errors will not cause the shell to exit.
20. If the `cdspell` shell option is enabled, the shell will attempt simple spelling correction for directory arguments to the `cd` builtin (see the description of the `cdspell` option to the `shopt` builtin in Section 4.3.2 [The Shopt Builtin], page 78). The `cdspell` option is only effective in interactive shells.
21. The shell will check the value of the `TMOU` variable and exit if a command is not read within the specified number of seconds after printing `$PS1` (see Section 5.2 [Bash Variables], page 87).

6.4 Bash Conditional Expressions

Conditional expressions are used by the `[[` compound command (see Section 3.2.5.2 [Conditional Constructs], page 12) and the `test` and `[` builtin commands (see Section 4.1 [Bourne Shell Builtins], page 52). The `test` and `[` commands determine their behavior based on the number of arguments; see the descriptions of those commands for any other command-specific actions.

Expressions may be unary or binary, and are formed from the primaries listed below. Unary expressions are often used to examine the status of a file or shell variable. Binary operators are used for string, numeric, and file attribute comparisons.

Bash handles several filenames specially when they are used in expressions. If the operating system on which Bash is running provides these special files, Bash uses them; otherwise

<code>\\$</code>	If the effective uid is 0, #, otherwise \$.
<code>\nnn</code>	The character whose ASCII code is the octal value <i>nnn</i> .
<code>\\</code>	A backslash.
<code>\[</code>	Begin a sequence of non-printing characters. This could be used to embed a terminal control sequence into the prompt.
<code>\]</code>	End a sequence of non-printing characters.

The command number and the history number are usually different: the history number of a command is its position in the history list, which may include commands restored from the history file (see Section 9.1 [Bash History Facilities], page 168), while the command number is the position in the sequence of commands executed during the current shell session.

After the string is decoded, it is expanded via parameter expansion, command substitution, arithmetic expansion, and quote removal, subject to the value of the `promptvars` shell option (see Section 4.3.2 [The Shopt Builtin], page 78). This can have unwanted side effects if escaped portions of the string appear within command substitution or contain characters special to word expansion.

6.10 The Restricted Shell

If Bash is started with the name `rbash`, or the `--restricted` or `-r` option is supplied at invocation, the shell becomes *restricted*. A restricted shell is used to set up an environment more controlled than the standard shell. A restricted shell behaves identically to `bash` with the exception that the following are disallowed or not performed:

- Changing directories with the `cd` builtin.
- Setting or unsetting the values of the `SHELL`, `PATH`, `HISTFILE`, `ENV`, or `BASH_ENV` variables.
- Specifying command names containing slashes.
- Specifying a filename containing a slash as an argument to the `.` builtin command.
- Using the `-p` option to the `.` builtin command to specify a search path.
- Specifying a filename containing a slash as an argument to the `history` builtin command.
- Specifying a filename containing a slash as an argument to the `-p` option to the `hash` builtin command.
- Importing function definitions from the shell environment at startup.
- Parsing the value of `SHELLOPTS` from the shell environment at startup.
- Redirecting output using the `>`, `>|`, `<>`, `>&`, `&>`, and `>>>` redirection operators.
- Using the `exec` builtin to replace the shell with another command.
- Adding or deleting builtin commands with the `-f` and `-d` options to the `enable` builtin.
- Using the `enable` builtin command to enable disabled shell builtins.
- Specifying the `-p` option to the `command` builtin.
- Turning off restricted mode with `'set +r'` or `'shopt -u restricted_shell'`.

These restrictions are enforced after any startup files are read.

When a command that is found to be a shell script is executed (see Section 3.8 [Shell Scripts], page 50), **rbash** turns off any restrictions in the shell spawned to execute the script.

The restricted shell mode is only one component of a useful restricted environment. It should be accompanied by setting **PATH** to a value that allows execution of only a few verified commands (commands that allow shell escapes are particularly vulnerable), changing the current directory to a non-writable directory other than **\$HOME** after login, not allowing the restricted shell to execute shell scripts, and cleaning the environment of variables that cause some commands to modify their behavior (e.g., **VISUAL** or **PAGER**).

Modern systems provide more secure ways to implement a restricted environment, such as **jails**, **zones**, or **containers**.

6.11 Bash and POSIX

6.11.1 What is POSIX?

POSIX is the name for a family of standards based on Unix. A number of Unix services, tools, and functions are part of the standard, ranging from the basic system calls and C library functions to common applications and tools to system administration and management.

The POSIX Shell and Utilities standard was originally developed by IEEE Working Group 1003.2 (POSIX.2). The first edition of the 1003.2 standard was published in 1992. It was merged with the original IEEE 1003.1 Working Group and is currently maintained by the Austin Group (a joint working group of the IEEE, The Open Group and ISO/IEC SC22/WG15). Today the Shell and Utilities are a volume within the set of documents that make up IEEE Std 1003.1-2024, and thus the former POSIX.2 (from 1992) is now part of the current unified POSIX standard.

The Shell and Utilities volume concentrates on the command interpreter interface and utility programs commonly executed from the command line or by other programs. The standard is freely available on the web at <https://pubs.opengroup.org/onlinepubs/9799919799/utilities/contents.html>.

Bash is concerned with the aspects of the shell's behavior defined by the POSIX Shell and Utilities volume. The shell command language has of course been standardized, including the basic flow control and program execution constructs, I/O redirection and pipelines, argument handling, variable expansion, and quoting.

The *special* builtins, which must be implemented as part of the shell to provide the desired functionality, are specified as being part of the shell; examples of these are **eval** and **export**. Other utilities appear in the sections of POSIX not devoted to the shell which are commonly (and in some cases must be) implemented as builtin commands, such as **read** and **test**. POSIX also specifies aspects of the shell's interactive behavior, including job control and command line editing. Only vi-style line editing commands have been standardized; emacs editing commands were left out due to objections.

6.11.2 Bash POSIX Mode

Although Bash is an implementation of the POSIX shell specification, there are areas where the Bash default behavior differs from the specification. The Bash *posix mode* changes the Bash behavior in these areas so that it conforms more strictly to the standard.